

Why RAG Evaluation Matters

A RAG system can fail at retrieval, grounding, or the answer itself.

01

User Question

02

Retrieval

03

Context

04

LLM

05

Answer

RETRIEVAL FAILURE

Relevant information was not retrieved.

GENERATION / GROUNDING FAILURE

The LLM adds information not supported by retrieved context.

ANSWER FAILURE

The response does not correctly answer the user's question.

Key message: measure both retrieval quality and answer quality.

THE FRAMEWORK

Our Lightweight RAG Evaluation Framework

Three explainable metrics implemented directly in the demo.



Retrieval



Context



LLM



Answer



Evaluation

ANSWER CORRECTNESS

Does the generated answer match the expected answer?

FAITHFULNESS

Is the generated answer supported by retrieved context?

RETRIEVAL QUALITY

Did vector search retrieve semantically relevant chunks?

1. Answer Correctness

Does the generated answer match the expected answer?

01

Normalize actual

Normalize the actual answer.

02

Normalize expected

Normalize the expected answer.

03

Compare

For answerable questions, check whether the expected answer appears in the actual answer.

04

PASS / FAIL

For unanswerable questions, check for “I could not find that information.”

EXAMPLE · PASS

Complaint response time: the expected answer says the bank aims to provide a final response within 15 business days; the generated answer says the same.

CAVEAT

Simple string-based correctness check designed for the demo.

2. Faithfulness — Is the Answer Grounded?

01

Normalize

Answer and context.

02

Extract

Unique words from answer and context.

03

Intersect

Find the shared words.

04

Score

Overlapping answer words ÷ total answer words.

05

PASS / FAIL

Score $\geq 0.50 \rightarrow$ PASS; score $< 0.50 \rightarrow$ FAIL.

EXAMPLE · GROUNDED

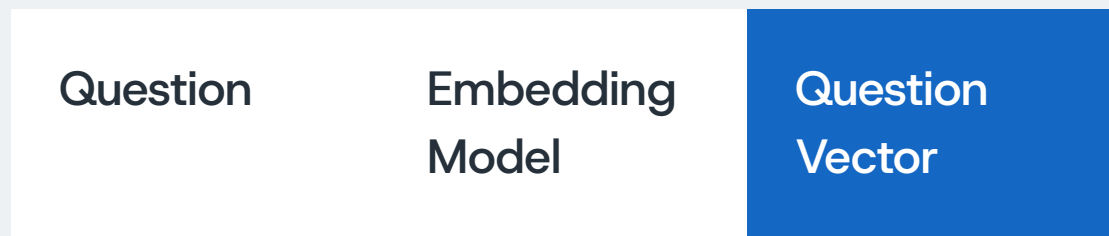
“The bank aims to provide a final response within 15 business days” vs. “The bank provides a final response within 15 business days.” Significant vocabulary overlap indicates grounding.

CAVEAT

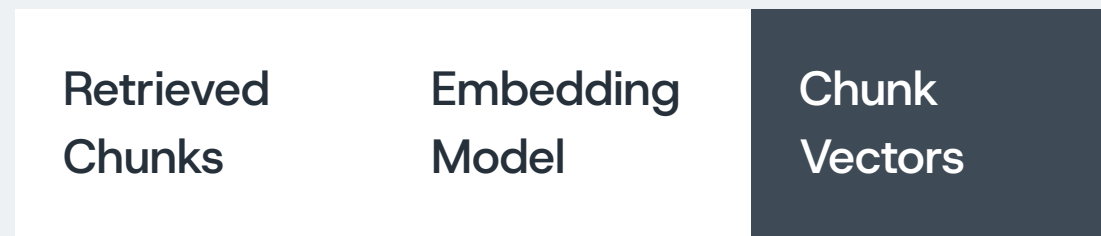
Lightweight word-overlap heuristic, not semantic or LLM-as-a-judge evaluation.

3. Retrieval Quality — Did We Find the Right Context?

QUESTION PATH



CONTEXT PATH



01

Compare

Dot-product similarity:
question vector vs. each
retrieved chunk vector.

02

Score

Similarity score for every
retrieved chunk.

03

Select

Highest similarity score.

04

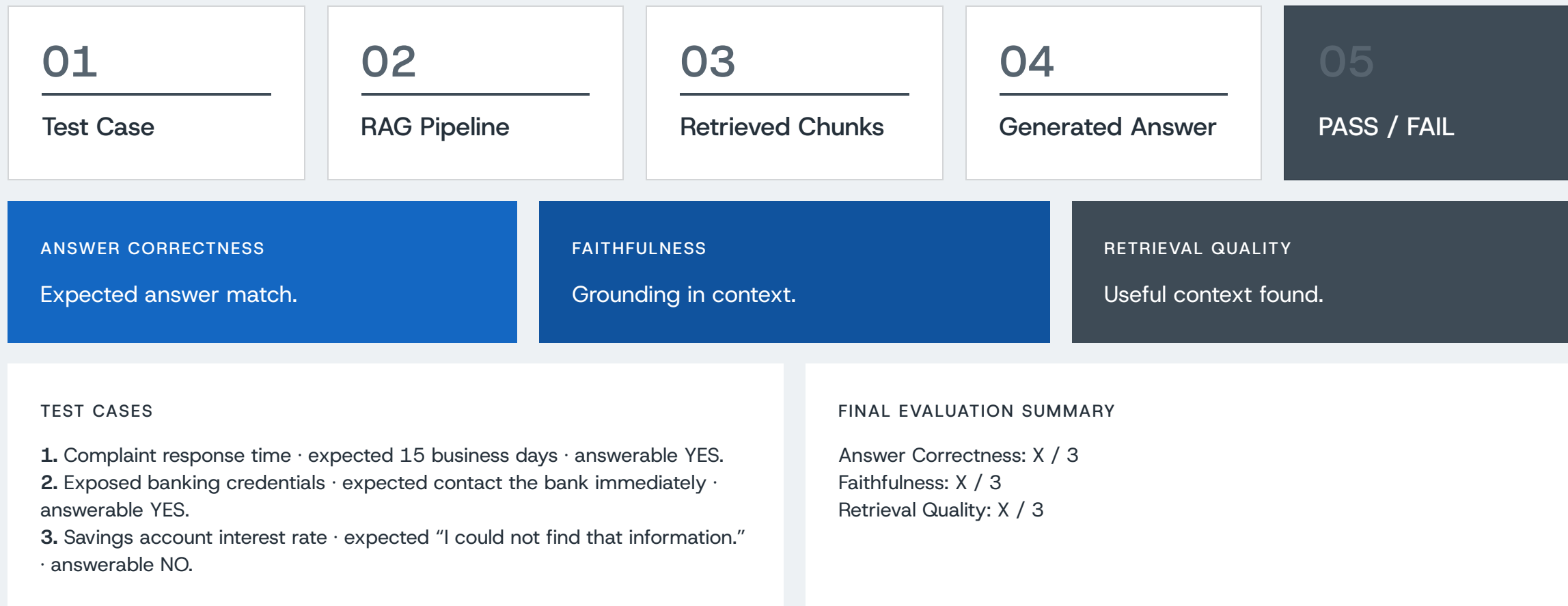
PASS / FAIL

Similarity $\geq 0.40 \rightarrow$ PASS;
< 0.40 $\rightarrow$ FAIL.

Caveat: 0.40 is for demonstration and needs calibration on a real production dataset.

Putting It All Together

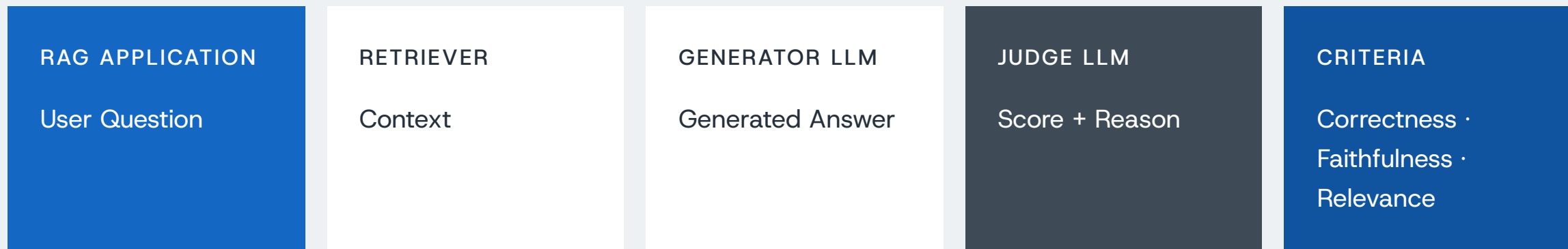
One test case runs through the RAG pipeline, then all three metric checks.



Takeaway: retrieval quality checks useful context, faithfulness checks grounding, and answer correctness checks whether the response satisfies the expected answer. Together they provide a simple first-level health check.

LLM-as-a-Judge: Using an LLM to Evaluate an LLM Application

Generator LLM creates the answer. Judge LLM evaluates the answer.



WHAT IT DOES

A separate LLM evaluation step assesses a generated response against a defined criterion, reference answer, or retrieved context.

EXAMPLE

Context: "The bank aims to provide a final response within 15 business days."

Answer: "The bank guarantees a response within 5 business days."

Judge: FAIL — contradicts retrieved context.

Lightweight Evaluation vs LLM-as-a-Judge

Two useful levels of evaluation — with different cost, speed, and semantic depth.

Our Current Demo

RAG → Groq Generator → Answer → Deterministic Python Evaluators

Metrics

Answer Correctness · expected-answer string matching

Faithfulness · answer/context word overlap

Retrieval Quality · embedding similarity + threshold

Advantages · Simple · Fast · Deterministic · Low cost · Easy to debug · Easy to explain

Limitations · Limited semantic understanding · wording-sensitive · simple thresholds · weak on complex claims

LLM-as-a-Judge

RAG → Generator LLM → Answer → Judge LLM → Score + Explanation

Metrics

Answer Correctness · Faithfulness · Relevance

Advantages · Better semantic understanding · paraphrase-aware · evaluates complex reasoning · provides explanations

Limitations · Additional cost and latency · judge can make mistakes · requires careful prompts and calibration

Takeaway: Our demo deliberately uses lightweight deterministic evaluation. LLM-as-a-Judge is a natural next step for more sophisticated evaluation — not an automatic guarantee of accuracy.

LLM-as-a-Judge for Faithfulness

Faithfulness is about factual support, not simply vocabulary overlap.

RETRIEVED CONTEXT

"The bank aims to provide a final response to standard complaints within 15 business days."

GENERATED ANSWER

"The bank responds to complaints within 15 business days and guarantees compensation if the response is delayed."

CLAIM EXTRACTION

Claim 1 · responds within 15 business days
Claim 2 · guarantees compensation if delayed

JUDGE LLM

Check each claim against context

SUPPORTED

Claim 1: Supported

UNSUPPORTED

Claim 2: Not supported

WHY IT MATTERS

Claim-level evaluation can detect unsupported information

CURRENT DEMO → PRODUCTION

The demo uses word overlap as a lightweight

From Lightweight Evaluation to Production Evaluation

Evaluation should evolve with the maturity of the RAG system.

Level 1

LIGHTWEIGHT DEMO

String Matching
Word Overlap
Embedding Similarity

Level 2

SEMANTIC EVALUATION

LLM-as-a-Judge
Claim Verification
Semantic Answer Evaluation

Level 3

PRODUCTION EVALUATION

Large Dataset · Multiple Metrics · Threshold
Calibration · Regression Testing · Continuous
Evaluation · Human Validation

EVALUATION MATRIX

Retrieval — did we retrieve relevant information?
Faithfulness — is the answer grounded in retrieved information?
Answer Correctness — does the answer address the question?
Answer Relevance — is the answer focused and useful?
Safety / Compliance — does the response follow required policies?

POC

Fast
Simple
Explainable

PRODUCTION

Semantic
Comprehensive
Calibrated
Continuous